TP Integrador: extractor de entidades en noticias

Seminario de Actualización · Tec. Sup. en Cs. de Datos e IA · IFTS N.º 18 · Ciclo lectivo 2026 · Docente: Cynthia M. Villagra

Integrantes: Matias Rodriguez; Lauger Natera.

Entrega: viernes 16/10/2026

A. El problema

1. ¿Qué problema querés resolver y para quién?

¿Por qué un modelo de IA y no unas reglas escritas a mano (un if con palabras clave)?

Una noticia tiene muchos nombres propios, y no es fácil ver de un vistazo quiénes aparecen, dónde ocurre todo y qué organizaciones intervienen. Nuestra app recibe el texto de una noticia en español y devuelve las personas, los lugares y las organizaciones que menciona. Está pensada para lectores que quieren ver rápido quién y dónde aparece en lo que leen.

Con reglas haría falta una lista de todos los nombres posibles de personas, ciudades, clubes y empresas, que nunca estaría completa y se desactualiza. Además, el mismo nombre puede ser cosas distintas según el contexto: en nuestras pruebas, "Mercedes" aparecía como persona y como ciudad en la misma frase, y los dos modelos las distinguieron por el contexto. E "Inter Miami", que no estaba en ninguna lista armada por nosotros, el modelo lo reconoció como organización con un puntaje de 0,9987.

B. El modelo

2. ¿Qué modelos consideraste y por qué te quedaste con uno?

Citá lo que leíste en su model card: tarea, etiquetas, idiomas, datos de entrenamiento, tamaño de los pesos.

Buscamos en Hugging Face modelos de reconocimiento de entidades (NER) filtrando por la tarea "Token classification" y por "Inferencia disponible". Entre los resultados, descartamos los modelos médicos y los de otros idiomas, y nos quedamos con dos multilingües que incluyen español, que comparamos con las mismas frases en la notebook:

	Babelscape/wikineural-multilingual-ner	**Davlan/xlm-roberta-base-ner-hrl**
Tarea	NER (clasificación de tokens)	NER (clasificación de tokens)
Etiquetas	la card no las lista; en nuestras pruebas devolvió PER, LOC, ORG y MISC	PER, LOC y ORG (con B- e I- para inicio y continuación)
Idiomas	9 según el texto de la card (de, en, es, fr, it, nl, pl, pt, ru); la etiqueta de la página dice 10	10 (ar, de, en, es, fr, it, lv, nl, pt, zh)
Datos de entrenamiento	WikiNEuRal, armado automáticamente a partir de Wikipedia (datos "silver", sin revisión manual)	para español, CoNLL 2002 (noticias); un dataset distinto por idioma
Modelo base	mBERT, ajustado 3 épocas	XLM-RoBERTa base
Tamaño de los pesos	709 MB (model.safetensors)	1,11 GB (model.safetensors)
Licencia	cc-by-nc-sa-4.0 (solo uso no comercial)	afl-3.0

Las dos cards advierten límites. La de Babelscape dice que, como se entrenó con Wikipedia, puede no generalizar bien las noticias. La de Davlan dice que se entrenó con artículos de noticias de un período específico y que puede no generalizar a otros dominios.

Nos quedamos con **Babelscape**. En nuestras pruebas con 9 frases, con un criterio estricto los dos acertaron 44 de 52 entidades. Si se acepta que el límite del texto cambie cuando el tipo es correcto, Babelscape llega a 46 y Davlan a 45, y la diferencia es un solo caso: Davlan marcó "Universidad Nacional de La Plata" como lugar (0,773) en vez de organización, y Babelscape acertó (0,960). Es decir que la precisión casi no los separó, y la advertencia de su card sobre las noticias no se tradujo en peor rendimiento. Pesó además que es más liviano (709 MB contra 1,11 GB, lo que deja más margen de memoria en el plan gratuito) y que corrió sin instalar librerías extra, mientras que Davlan exigió instalar sentencepiece, tiktoken y protobuf. A cambio, Davlan fue más rápido (0,132 s contra 0,185 s por frase).

3. ¿Dónde se equivoca?

Mostrá ejemplos reales de tu notebook y explicá por qué creés que se equivoca, relacionándolo con los datos con los que fue entrenado.

Probamos los dos modelos con 9 frases (6 de noticias comunes y 3 pensadas para que fallaran), con 52 entidades esperadas en total. Cada modelo acertó 44, o 46 y 45 si se acepta que el límite del texto cambie cuando el tipo es correcto. Los errores se concentran en tres casos:

- **Jerga y minúsculas:** En "che, el sábado fui a la cancha de boca con el nico y la vale, y después nos fuimos a comer a palermo...", ningún modelo marcó ninguna de las 5 entidades (boca, nico, vale, palermo, obelisco). Son 10 de los 16 errores. Creemos que se debe a que ambos se entrenaron con texto bien escrito, donde la mayúscula inicial es una pista fuerte de nombre propio, y donde apodos como "nico" o "vale" no aparecen como nombres.
- **Equipos, eventos y límites del texto (frases 5, 6 y 7).** Los dos modelos marcaron "Pumas" sin la palabra "Los". Babelscape marcó "provincia de Buenos Aires" completo como un solo lugar, con el tipo correcto pero otro límite y "Feria del Libro de Buenos Aires" la marcó como MISC Babelscape y no la detectó Davlan, que según su model card solo tiene PER, LOC y ORG y no tenía una categoría para eventos.
- **Organizaciones con una ciudad adentro (frase 3).** Davlan marcó "Universidad Nacional de La Plata" como lugar (0,773) en vez de organización, y Babelscape acertó (0,960). Creemos que el nombre de la ciudad pesó más que el resto.

Sobre los datos de entrenamiento: Babelscape se entrenó con Wikipedia, y su model card advierte que puede no generalizar bien a otros géneros como las noticias. Davlan se entrenó con noticias de un período específico, y su card advierte que puede no generalizar a otros dominios. En la frase de jerga, que está lejos de ese tipo de texto, fallaron los dos.

Lo que no es un error: ninguno de los dos marca fechas, cifras ni cargos, porque solo fueron entrenados con personas, lugares y organizaciones.

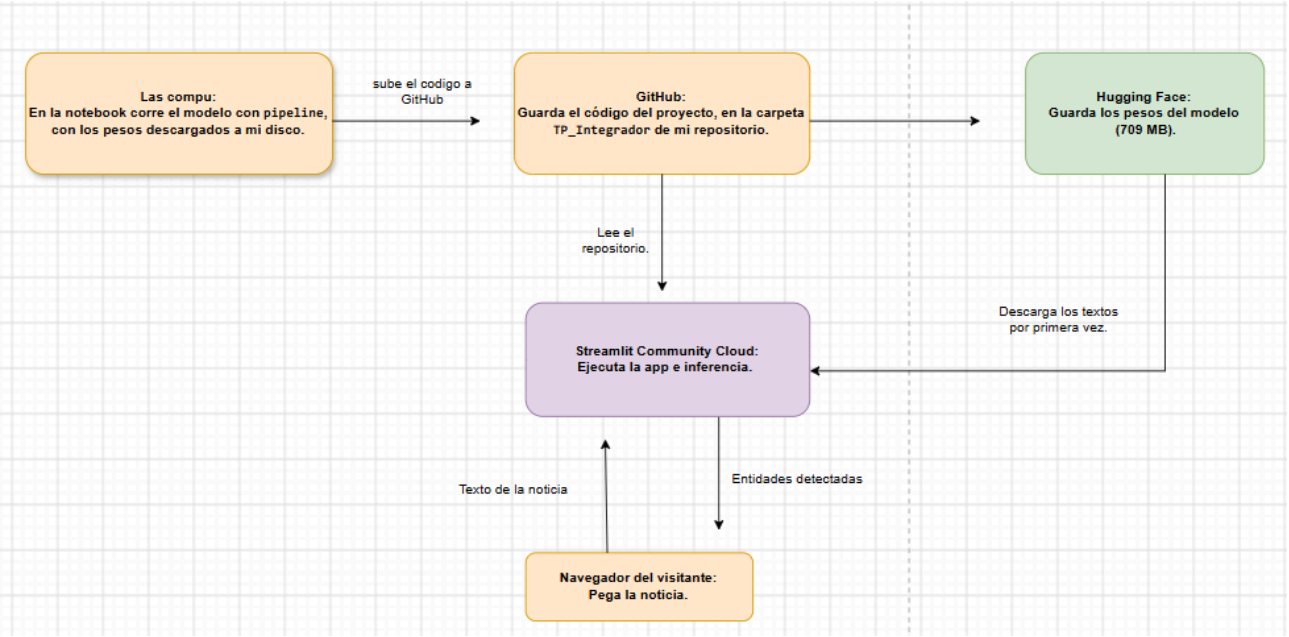
4. Tu app, ¿está entrenando o infiriendo? Explicá la diferencia con tu propio caso.

Entrenar es enseñarle al modelo con muchos ejemplos ya etiquetados, hasta que aprende a reconocer personas, lugares y organizaciones. Es lo que hicieron los autores de Babelscape con WikiNEuRal, un conjunto de datos armado a partir de Wikipedia. Inferir es usar un modelo que ya aprendió para analizar un texto nuevo.

Nuestra app infiere. Cuando alguien pega una noticia y aprieta "Analizar", la app llama al modelo ya entrenado (resultado = ner(texto)) y muestra lo que detecta. No modifica el modelo ni aprende de los textos que recibe. En nuestro caso, los pesos (709 MB) son los mismos que publicaron los autores, y nosotros solo los descargamos desde Hugging Face y los usamos. En ningún momento del proyecto entrenamos nada.

C. Dónde corre cada cosa

5. ¿Dónde vive el código y dónde vive el modelo? Hacé un esquema simple (cajas y flechas, a mano o en cualquier herramienta) con los lugares que intervienen en tu app: tu compu, GitHub, la plataforma de deploy y Hugging Face. En cada caja anotá qué hay ahí, y marcá con flechas qué viaja de un lugar a otro cuando alguien usa la app. Tiene que quedar claro: ¿dónde está guardado tu código? ¿Dónde se ejecuta? ¿Dónde están los pesos del modelo y dónde se hace la inferencia?



Qué hay en cada lugar:

- **Nuestras computadoras:** escribimos y probamos el código (notebook.ipynb, app.py, requirements.txt y README.md). En la notebook corrimos el modelo con pipeline, con los pesos descargados a nuestro disco.
- **GitHub:** guarda el código del proyecto, en la carpeta TP_Integrador del repo de la cursada.
- **Streamlit Community Cloud:** es la plataforma de deploy. Lee el repo, instala las librerías de requirements.txt (streamlit, transformers y torch), ejecuta app.py y tiene el modelo cargado en memoria.
- **Hugging Face:** guarda los pesos del modelo (709 MB).
- **Navegador del visitante:** la persona entra al link y pega el texto de una noticia.

Qué viaja entre lugares, cuando alguien usa la app: subimos el código desde nuestras computadoras a GitHub, y Streamlit lo lee de ahí (y se actualiza solo cuando cambia el repo). La primera vez, Streamlit descarga los pesos desde Hugging Face. Después, el visitante envía el texto de la noticia a Streamlit, y Streamlit le devuelve las entidades detectadas.

Respondiendo directo a las cuatro preguntas:

- **¿Dónde está guardado nuestro código?** En GitHub, con una copia en nuestras computadoras.
- **¿Dónde se ejecuta?** En los servidores de Streamlit Community Cloud, no en nuestras computadoras ni en GitHub.
- **¿Dónde están los pesos del modelo?** Guardados en Hugging Face. El servidor de Streamlit descarga una copia para poder usarlos.
- **¿Dónde se hace la inferencia?** En Streamlit, porque usamos pipeline. Hugging Face solo aloja los pesos. Si hubiéramos usado InferenceClient, la inferencia se haría en los servidores de Hugging Face y el modelo no se descargaría.

6. ¿Usaste pipeline o InferenceClient en la app publicada?

Justificalo con números. ¿Qué pasaría si hubieras elegido la otra?

En la app publicada usamos pipeline: el modelo se descarga desde Hugging Face y corre dentro de Streamlit Community Cloud.

Los números que respaldan la decisión:

- **Peso del modelo:** 709 MB (model.safetensors de Babelscape).

- **Memoria del plan gratuito:** Streamlit Community Cloud indica entre 690 MB y 2,7 GB. Los pesos solos ya superan el mínimo y la app funciona, por lo que el límite real para esta app es mayor.
- Tiempo de respuesta con pipeline, medido en una de nuestras computadoras con las 9 frases de prueba: 0,185 s por frase en promedio (máximo 0,377 s). La primera ejecución, que incluye la descarga del modelo, tardó 1 min 37 s. No medimos el tiempo dentro de Streamlit.

Con InferenceClient no pudimos medir nada: al llamarlo recibimos el error 402 Payment Required ("You have no remaining credits"). Según la documentación de Hugging Face, el plan gratuito incluye solo un crédito mensual de $0,10.

¿Qué pasaría si hubiéramos elegido la otra? Con InferenceClient la app no descargaría el modelo, así que debería gastar menos memoria y arrancar más rápido. Pero cada consulta viajaría por internet y gastaría créditos de la cuenta dueña del token: con el plan gratuito dejaría de funcionar al agotarse, y con muchos visitantes se agotaría enseguida. Elegimos pipeline porque no depende de créditos y el modelo entra en la memoria del plan gratuito, a cambio de una primera carga más lenta.

D. El deploy

7. ¿Dónde deployaste y por qué ahí?

Compará tu elección con al menos otras dos opciones vistas en clase (hosting compartido, VPS, PaaS, serverless, nubes elásticas) y explicá por qué las descartaste. ¿Por qué tu app necesita un servidor que quede encendido?

Publicamos la app en Streamlit Community Cloud, una plataforma de tipo PaaS: la plataforma se encarga del servidor y nosotros solo aportamos el código. La elegimos por tres motivos: es gratuita, se conecta directo con nuestro repo de GitHub y se actualiza sola con cada cambio, y su memoria alcanza para un modelo de 709 MB. Además, publicar fue directo: la app mínima funcionó al primer intento.

Las otras opciones que comparamos y por qué las descartamos:

- **Render (PaaS):** su instancia gratuita tiene 512 MB de RAM y 0,1 CPU, y el modelo solo (709 MB) ya supera esa memoria.

- **Hugging Face Spaces:** según la documentación actual, un Space que ejecuta código requiere un plan pago, y la propia consigna aclara que depende de la cuenta.

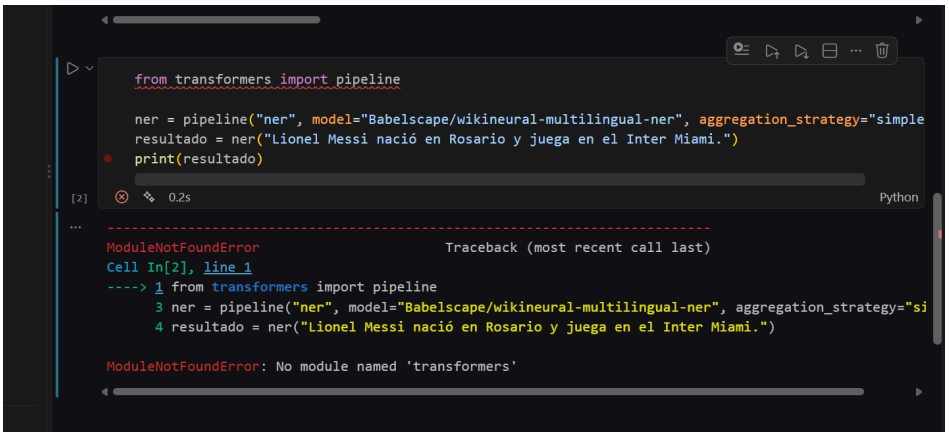
¿Por qué la app necesita un servidor encendido? Porque la app es un programa que tiene que estar esperando pedidos: cuando alguien abre el link y aprieta "Analizar", el código tiene que estar corriendo y el modelo cargado en memoria para responder en ese momento. Si el servidor estuviera apagado, nadie podría usarla. En el plan gratuito la app puede dormirse si nadie la usa, y tarda en volver a despertar.

E. Lo que se rompió

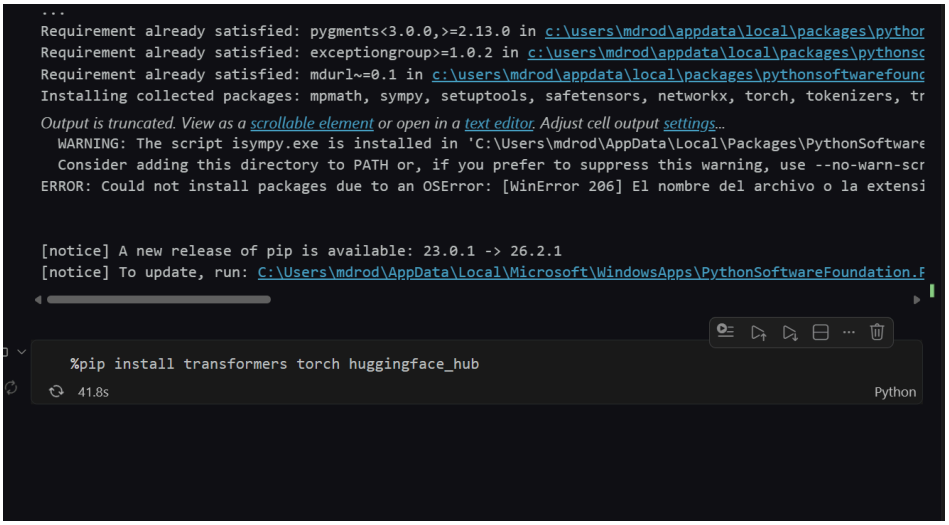
8. Documentá al menos dos errores reales que te aparecieron durante el trabajo. Para cada uno: el mensaje de error completo (captura o texto copiado); dónde apareció: en tu compu, en la plataforma de deploy o en Hugging Face; qué lo causaba; cómo lo resolviste, paso a paso, como para que otra persona pueda seguirlo.

Error 1: no se encontraba el módulo transformers

- **Mensaje:** ModuleNotFoundError: No module named 'transformers', al ejecutar "from transformers import pipeline". Antes, la celda de instalación con !pip había terminado con un error de instalación: ERROR: Could not install packages due to an OSError: [WinError 206] El nombre del archivo o la extensión es demasiado largo.



Error 1: ModuleNotFoundError al importar transformers en la notebook.



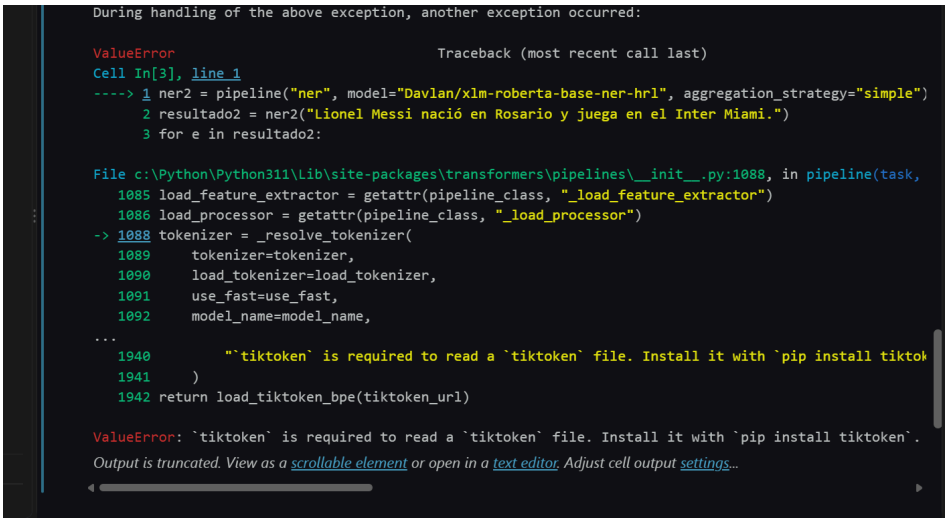
Error 1: salida de la instalación con !pip, con el error WinError 206.

- **Dónde apareció:** en nuestras computadoras, en la notebook de VS Code.
- **Qué lo causaba:** la instalación con !pip se hizo en un Python de la Microsoft Store (ruta ...\WindowsApps\PythonSoftwareFoundation...) y falló por el largo de la ruta de archivos (WinError 206). Además, la notebook usaba otro Python (c:\Python\Python311), donde transformers no estaba instalado. El tilde verde de la celda no avisaba del problema: solo indica que la celda terminó de ejecutarse, no que la instalación haya salido bien.
- **Cómo lo resolvimos:**
  1. En una celda, ejecutar %pip install transformers torch huggingface_hub (con %, que instala en el Python que usa la notebook).
  2. Arriba a la derecha, en "Seleccionar el kernel", elegir Python 3.11 (c:\Python\Python311).
  3. reiniciar y Volver a ejecutar la celda del pipeline.

  Resultado: la instalación terminó y el modelo funcionó.

Error 2: faltaban librerías para el tokenizador del modelo de Davlan

- **Mensaje (líneas principales):** "[transformers] Could not extract SentencePiece model from ...", "ValueError: tiktoken is required to read a tiktoken file. Install it with pip install tiktoken."



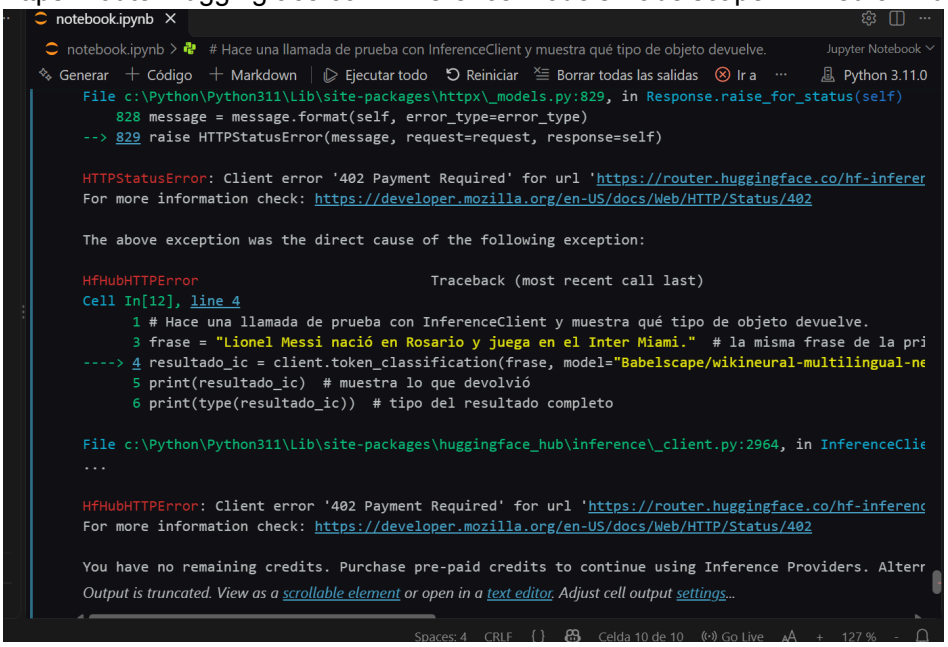
Error 2: final del mensaje, ValueError por falta de tiktoken.

- **Dónde apareció:** en nuestras computadoras, en la notebook, al cargar con pipeline el modelo de Davlan (ya descargado desde Hugging Face).
- **Qué lo causaba:** el tokenizador de ese modelo (la parte que convierte el texto en números) se guarda como sentencepiece.bpe.model y el repositorio no trae tokenizer.json. Para leerlo, transformers necesita la librería sentencepiece. Como no estaba instalada, probó con tiktoken, que tampoco estaba. Babelscape no tuvo el problema porque su repositorio sí trae un tokenizer.json.
- **Cómo lo resolvimos:**
  1. En una celda, ejecutar %pip install sentencepiece tiktoken protobuf.
  2. Reiniciar y Volver a ejecutar la celda del import de pipeline y después la del modelo de Davlan.

  Resultado: la celda corrió (5,0 s) y el modelo detectó PER, LOC y ORG. Instalamos las tres librerías juntas, así que no sabemos cuál fue la decisiva; el propio mensaje apunta a sentencepiece y tiktoken.

Error 3: 402 Payment Required al usar InferenceClient (límite que no se resolvió)

- **Mensaje:** HfHubHTTPError: Client error '402 Payment Required' for url 'https://router.huggingface.co/hf-inference/models/Babelscape/wikineural-multilingual-ner'.



Error 3: 402 Payment Required al llamar a InferenceClient en la notebook.

- **Dónde apareció:** en nuestras computadoras, en la notebook, pero el rechazo vino de los servidores de Hugging Face.
- **Qué lo causaba:** la cuenta no tenía créditos de Inference Providers. Según la documentación de Hugging Face, el plan gratuito incluye solo un crédito mensual de 0,10 dólares.
- **Cómo lo resolvimos:** no se resolvió. El mensaje indica que hace falta comprar créditos prepagos o suscribirse a PRO, y optamos por usar pipeline en la app publicada (ver pregunta 6).

9. Límites de tu solución: ¿qué pasa si la usan 1000 personas a la vez? ¿Si Hugging Face cambia las condiciones del plan gratuito (como ya pasó con los Spaces)? ¿Qué cambiarías para que sea un producto real?

Si la usan 1000 personas a la vez. Nuestra app corre en un solo servidor gratuito, con memoria limitada, y el modelo se carga una sola vez para todos los visitantes. En nuestras pruebas, cada consulta tardó 0,185 s en promedio en una computadora, si las 1000 consultas se atendieran una tras otra, la última esperaría unos 3 minutos aprox, y eso sin contar que no medimos el tiempo dentro de Streamlit. Es probable que las respuestas se demoren mucho o que la plataforma corte la app por superar sus límites de recursos: la propia plataforma tiene un aviso para ese caso ("this app has gone over its resource limits").

Si Hugging Face cambia las condiciones del plan gratuito. Ya nos pasó algo parecido: el plan gratuito de InferenceClient incluye solo 0,10 dólares de crédito mensual y nos quedamos sin poder usarlo. Nuestra app no depende de Hugging Face para responder, pero sí para descargar los pesos del modelo cuando arranca. Si ese acceso cambiara (por ejemplo, si exigiera una cuenta o limitara las descargas), la app podría no poder cargar el modelo al reiniciarse. Lo mismo vale para Streamlit: las cifras de memoria que usamos son de 2024 y la plataforma aclara que pueden cambiar.

Qué cambiaríamos para que sea un producto real:

- **Modelo:** Babelscape tiene licencia no comercial (cc-by-nc-sa-4.0), así que haría falta un modelo con una licencia que permita uso comercial, o pedir permiso a sus autores.
- **Evaluación:** probamos con 9 frases y 52 entidades; antes de usarla en serio habría que evaluarla con muchas más noticias reales.
- **Lo que no detecta:** no marca fechas, cifras ni cargos, y falló con texto informal y en minúsculas; habría que sumar otro modelo o reglas, y ajustar el modelo con ese tipo de texto.
- **Infraestructura:** un plan pago con memoria y CPU garantizadas, que no se duerma y que pueda crecer según la cantidad de visitantes, y una copia propia de los pesos del modelo para no depender de la descarga desde Hugging Face.

F. Vocabulario

10. Tomá 5 líneas de tu app.py y señalá en ellas: una clase, una instancia, un método, un atributo, un parámetro y su argumento, y un valor de retorno.

En nuestro código no tenemos clases declaradas explícitamente, sin embargo estas cinco lineas expresan lo solicitado:

Línea 1°

```
return pipeline("ner", model="Babelscape/wikineural-multilingual-ner", aggregation_strategy="simple")
```

Esta línea señala un parámetro, un argumento y un valor de retorno. “Model” es el parámetro y el “Babelscape/wikineural-multilingual-ner” el argumento y la llamada “pipeline” produce el valor que devuelve la fincion “cargar_modelo()”.

Línea 2°

```
texto = st.text_area("Texto de la noticia", height=200)
```

Esta línea muestra “texto” que es una instancia de “str”, height es un parametro de “text_area” y “200” el argumento.

Línea 3°

```
if not texto.strip():
```

En esta línea “strip” es un metodo de instancia “texto” .

Línea 4°

```
ner = cargar_modelo()
```

“ner” recibe la instancia del extractor que devuelve “cargar_modelo()”.

Línea 5°

```
st.write(f"{NOMBRES.get(e['entity_group'], e['entity_group'])}: {e['word']} ({e['score']:.2f})")
```

Y por último, en esta linea “write” se accede como atributo de “st” y se usa para mostrar el resultado.

Links

Link de la app:
https://matiasrodriguezseminario2026-8dnzgoc2y8kf54rythqt8v.streamlit.app/

Link del repositorio:
https://github.com/MatiasDamianRodriguez/MatiasRodriguezSeminario2026/tree/main/TP_Integrador

Uso de la IA generativa

Usamos Claude (de Anthropic) como ayuda durante el trabajo. Lo usamos para:

- Clarar la idea general del proyecto, de la app y consejos para buscar modelos candidatos en Hugging Face.
- Ayudar para explicar errores en el código en las celdas de la notebook y como solucionarlos.
- Ayudar al realizar la PPT para la presentación.